

CPAI: A Controlled-Point Discriminant Analysis for IoT Network Intrusion Detection

Van-Hoi Nguyen^{a,b,1}, Xuan-Ha Nguyen^{a,b,1}, Kim-Hung Le^{a,b,*}

^aUniversity of Information Technology, Ho Chi Minh City, Vietnam

^bVietnam National University, Ho Chi Minh City, Vietnam

Abstract

The proliferation of Internet of Things (IoT) devices poses significant security challenges, particularly intrusion detection, due to evolving cyber attacks and their resource constraints. While existing distance-based approaches like Null Foley-Sammon Transform (NFST) show promise, they suffer from singularity and small sample size problems, limiting their effectiveness in diverse IoT scenarios. In this paper, we introduce Controlled-Point Discriminant Analysis (CPAI), a novel algorithm designed for robust IoT network intrusion detection. CPAI addresses these limitations by effectively controlling the location of projected data points in the discriminant space and employing kernel tricks with dual training strategies - One-vs-All (OvA) for computational efficiency and One-vs-One (OvO) for enhanced accuracy. Extensive evaluation across five benchmark IoT network security datasets demonstrates CPAI's superiority in terms of accuracy and stability compared to KNFST and other distance-based algorithms. The CPAI-OvO variant achieves over 98% accuracy while consistently maintaining false positive rates below 1% across all datasets, highlighting its effectiveness and reliability.

Keywords: Linear discriminant analysis, Null Foley-Sammon transform, Feature extraction; Fisher criterion, Foley-Sammon transform, Singularities.

1. Introduction

The Internet of Things (IoT) has revolutionized various critical sectors (e.g., healthcare, logistics, and manufacturing) by enabling seamless communication and automation over the Internet. While this significantly improves the efficiency of operations and services, but it also introduces cybersecurity threats by exposing networks to unauthorized access, data breaches, and service disruptions [1]. Furthermore, the inherent IoT devices often have limited computational and energy resources, making them unsuitable for computationally intensive security mechanisms. Therefore, developing lightweight yet effective security solutions specifically designed for IoT systems is a critical necessity [2].

1.1. Paradigms in Anomaly-Based Intrusion Detection

Anomaly-based Network Intrusion Detection Systems (aNIDS) represent a promising solution for this challenge. Their primary strength lies in their adaptability to automatically identify threats by analyzing deviations from normal network behavior, making them more flexible than signature-based systems that rely on predefined attack patterns. Numerous studies have demonstrated the superior performance of aNIDS in conventional networks [1; 3; 4]. Within the domain of aNIDS, several methodological paradigms have been dominant.

Deep learning (DL) and tree-based algorithms are widely used in proposed aNIDSs due to their effectiveness in processing tabular network traffic data [1; 5; 6]. DL models excel at automatically capturing complex patterns [7]. However, they typically demand substantial computational resources (e.g., GPUs, significant memory), making them impractical for resource-constrained IoT devices. Even though inference can be optimized for edge devices (e.g., via model distillation), the initial training overhead remains a significant barrier. While tree-based models are generally more lightweight, they can struggle in unsupervised or semi-supervised settings common in IoT, where labeled data for novel attacks is limited.

Given these trade-offs, distance-based algorithms have emerged as a promising alternative for IoT network security. These algorithms do not require specialized hardware like GPUs and are highly adaptable in contexts where attack labels are unavailable, making them more suitable for resource-constrained IoT devices [8]. This has led to a renewed interest in methods rooted in Linear Discriminant Analysis (LDA) [9], which offer a powerful middle ground: they provide a principled approach to feature extraction that is both computationally efficient and more interpretable than deep learning "black boxes". Its key concept involves projecting instances from a high-dimensional space into a lower-dimensional subspace through linear transformation, thereby compacting samples and making them easier to classify.

1.2. The Challenge of Null-Space Discriminant Analysis

A state-of-the-art method in this direction is the Null Foley-Sammon Transform (NFST) [10], that extends classical Lin-

*Corresponding author

Email addresses: hoinv@uit.edu.vn (Van-Hoi Nguyen), hanx@uit.edu.vn (Xuan-Ha Nguyen), hunglk@uit.edu.vn (Kim-Hung Le)

¹These authors contributed equally to this work.

ear Discriminant Analysis [9; 11]. The theoretical elegance of NFST lies in its objective to project all samples of a class to a single, unique point by enforcing zero within-class scatter, thereby maximizing discrimination according to the Fisher criterion. Despite its strong theoretical appeal, our analysis and recent literature reveal two critical, unresolved flaws that severely hinder NFST’s reliability in practical IoT scenarios:

- *Singularity Problem:* The foundational assumption that projected class points remain distinct is not guaranteed. However, as demonstrated by Ali et al. [12], these singularities can erroneously overlap, leading to catastrophic misclassification. The problem gets worse in IoT networks where normal and malicious behaviors could have several similar network characteristics, reducing the model’s accuracy. To the best of our knowledge, this singularity problem remains unresolved within the application of aNIDS in IoT networks.
- *Small sample size problem:* The second theoretical need of NFST is the linearity of sample vectors. This assumption yields that the number of samples (n) does not exceed the feature dimensionality (d), i.e., $n \leq d$. Consequently, applying NFST in IoT networks may reduce accuracy or even make it impossible to find a solution.

1.3. Our Contribution: Controlled-Point Discriminant Analysis

Recognizing these critical limitations, we propose a Controlled-Point discriminant Analysis for IoT (CPAI), a novel framework designed to overcome the theoretical drawbacks of NFST while remains the strengths of discriminant analysis. The core innovation of CPAI is its ability to *explicitly control the placement of projected data points*, a principle that fundamentally resolves the singularity problem. Furthermore, by leveraging the kernel trick, CPAI robustly handles the non-linear, efficiently handling the constraint on the number of samples.

Crucially, the formulation of CPAI as a regularized least-squares problem admits a *closed-form analytical solution*. This transforms the learning problem into solving a single system of linear equations. This structural simplicity not only makes the model easier to analyze but also avoids potential issues like local minima and convergence problems inherent in more complex optimizers.

This computational efficiency, combined with its high accuracy, positions CPAI as a framework uniquely aligned with the core requirements of IoT security: *robustness, efficiency, and reliability*. To enhance its flexibility for diverse deployment needs, we propose two training strategies: CPAI-OvA for computational efficiency, and CPAI-OvO for superior accuracy. We conduct extensive experiments on five benchmark IoT datasets, demonstrating that our proposed CPAI framework significantly outperforms not only its predecessor (KNFST) but also a wide range of other strong baseline methods.

The remainder of this paper is organized as follows. Section 2 details the theoretical foundations and implementation of our proposed CPAI framework. A direct comparison between

CPAI and its predecessor, NFST, is provided in Appendix B to highlight our key theoretical advancements. Section 3 presents the comprehensive experimental setup, results, and analysis, validating CPAI’s superior performance against a wide range of competitors. Finally, Section 4 concludes the paper with a summary of our findings and discusses directions for future work. To ensure the paper is self-contained, an overview of the NFST algorithm is provided in Appendix A.

2. Our proposed approach

In this section, we present our proposed algorithm CPAI, detailing its theoretical foundations and implementation. We first introduce the key concepts underlying the algorithm, followed by a detailed description of its implementation. We then explain how we integrate kernel methods to overcome the small sample size problem, enhancing the approach’s suitability for real-world IoT network security applications.

2.1. Linear case

2.1.1. Theoretical description

Suppose the dataset consist of n samples $\{x^i\}_{i=1}^n$ belonging to c classes, denoted as $\{x_1^i\}_{i=1}^{n_1}, \dots, \{x_c^i\}_{i=1}^{n_c}$, where n_j is the number of samples in the j^{th} class, and $x_j^i \in \mathbb{R}^d$ represents the i^{th} sample vector of the j^{th} class. The total number of samples is $n = \sum_{j=1}^c n_j$.

This section primarily considers the small sample size case in the classical sense where the number of data instances is less than or equal to the feature dimension (i.e., $n \leq d$), and assumes linear independence among sample vectors, following prerequisites common in NFST literature. Recall that for arbitrary vector $x, y \in \mathbb{R}^d$, the inner product is defined as $(x, y) = x^T y = y x^T = (y, x)$, while the Euclidean norm or the length of vector x is $|x| = (x, x)^{\frac{1}{2}}$. The projection of the vector u onto a non-zero vector v is a scalar $(u, v)/|v|$.

Note that, in the small sample size case such as those encountered in face recognition, the assumption of *linear independence of the sample vectors* $\{x^i\}$ is often made. This hypothesis is a crucial requirement for NFST [10] and related studies [13; 14] and more recently in [12].

Theorem 1. Suppose $\{u_1, \dots, u_n\} \in \mathbb{R}^d$ are linearly independent. For any target projection vector $\beta \in \mathbb{R}^n$, there exists a unique vector $\theta \in \text{span}\{u_1, \dots, u_n\} := \{\gamma_1 u_1 + \gamma_2 u_2 + \dots + \gamma_n u_n, \gamma_i \in \mathbb{R}, i = 1, n\}$ such that $(\theta, u_i) = \beta_i$ for all $i = 1, n$.

Proof 1. For an arbitrary fixed target vector $\beta = (\beta_1, \dots, \beta_n)^T \in \mathbb{R}^n$. The proof ends if the linear equation below admits a unique solution θ within the subspace $\text{span}\{u_1, \dots, u_n\}$:

$$Q^T \theta = \beta, \quad (1)$$

where $Q = (u_1, u_2, \dots, u_n) \in \mathbb{R}^{d \times n}$. Since we seek a solution $\theta \in \text{span}\{u_1, \dots, u_n\}$, it can be expressed as a unique linear

combination of the basis vectors $\{u_1, \dots, u_n\}$ (as they are linearly independent)

$$\theta = \alpha_1 u_1 + \dots + \alpha_n u_n = Q\alpha, \quad (2)$$

where $\alpha = (\alpha_1, \dots, \alpha_n)^T \in \mathbb{R}^n$ is the unique vector of coefficients. Substituting (1) with its basis expansion (2), one obtains

$$Q^T Q \alpha = \beta. \quad (3)$$

The existence and uniqueness of the solution θ depend on the existence and uniqueness of the solution α to (3). It remains to prove that the $n \times n$ matrix $Q^T Q$ is of full rank (i.e., invertible). To this end, if α' is a solution to the homogeneous system of linear equations $Q^T Q \alpha' = 0$, then $(\alpha')^T Q^T Q \alpha' = (Q\alpha', Q\alpha') = 0$. This means $Q\alpha' = 0$. Since the columns of Q (the vectors $u_1, \dots, u_n$) are linearly independent by assumption, the only solution to $Q\alpha' = 0$ is the trivial solution $\alpha' = 0$. It follows from the algebraic theory that $Q^T Q$ is of full rank.

Consequently, for any given β , the system (3) has a unique solution α . Substituting this unique α back into (2) yields the unique solution θ within the subspace $\text{span}\{u_1, \dots, u_n\}$. It ends the proof.

This theorem guarantees that, under the condition of linear independence of the sample vectors $\{x^i\}$, we can find a unique discriminant vector θ (within the span of these samples) that maps each training sample x_j^i to an arbitrarily predefined target value β_j . This allows us to force samples of the j^{th} class to map to a single target value (e.g., β_j), while samples from different classes (i.e., $\ell \neq j$) can be mapped to different target values ($\beta_\ell \neq \beta_j$). Remarkably, these target projection values can be chosen arbitrarily.

Lemma 2. Let H_j denote an affine hull generated by the data samples of the j^{th} class i.e.,

$$H_j = \{x = a_1 x_j^1 + a_2 x_j^2 + \dots + a_{n_j} x_j^{n_j} : \sum_{i=1}^{n_j} a_i = 1\}.$$

Let θ be a vector constructed according to Theorem 1 such that $(\theta, x_j^i) = \beta_j$ (a constant value) for all samples $i = 1, \dots, n_j$ of the j^{th} class. Then, for any vector $x \in H_j$, it holds that $(\theta, x) = \beta_j$.

Proof 2. The proof is straightforward that since for any $x \in H_j$, one has the following linearly combination: $x = a_1 x_j^1 + \dots + a_{n_j} x_j^{n_j}$ for some coefficients $a_1, \dots, a_{n_j}$ such that $\sum_{i=1}^{n_j} a_i = 1$. By the linearity of the inner product, By construction of θ (based on Theorem 1 with appropriate β), we have $(\theta, x_j^i) = \beta_j$ for all $i = 1, \dots, n_j$. Substituting this into the equation above yields:

$$(\theta, x) = \sum_{i=1}^{n_j} a_i \beta_j = \beta_j \sum_{i=1}^{n_j} a_i$$

provided that $(\theta, x_j^i) = \beta_j$ for any $i = \overline{1, n_j}$, and $\sum_{i=1}^{n_j} a_i = 1$. It claims that $(\theta, x) = \beta_j$. The result holds.

The above lemma ensures that any point within the affine hull of the samples of j^{th} class maps to the same predefined target value β_j when projected onto the discriminant direction θ .

Based on Theorem (1), we can construct discriminant directions θ_i by choosing appropriate target vectors β . A transformation can then be defined using one or more such vectors. Let $\Phi = (\theta_1, \dots, \theta_n)$ be a matrix whose columns are n discriminant vectors. The transformed data is given by $y = \Phi^T x$. With appropriately chosen β vectors, this transformation maps samples of the same class to a single target point (or vector) in the n -dimensional projection space, while ensuring that target points for different classes are distinct. This leads to two primary implementation strategies for CPAI as a classifier:

1. **One-vs-All (OvA) approach:** This strategy aims to classify all c classes using a single discriminant vector θ . We set the target vector $\beta \in \mathbb{R}^n$ such that all samples belonging to the j^{th} class have the same target value β_j , and target values for different classes are distinct ($\beta_k \neq \beta_j$ for $k \neq j$). For example, one could set:

$$\beta = (\underbrace{\beta_1, \dots, \beta_1}_{n_1}, \dots, \underbrace{\beta_j, \dots, \beta_j}_{n_j}, \dots, \underbrace{\beta_c, \dots, \beta_c}_{n_c})^T, \quad (4)$$

where $\beta_1, \dots, \beta_c$ are distinct, arbitrarily chosen scalar values. This idea is illustrated in Figure 1, where samples from multiple classes are projected onto the single discriminant direction θ , mapping to distinct points $\beta_1, \beta_2, \dots, \beta_j$.

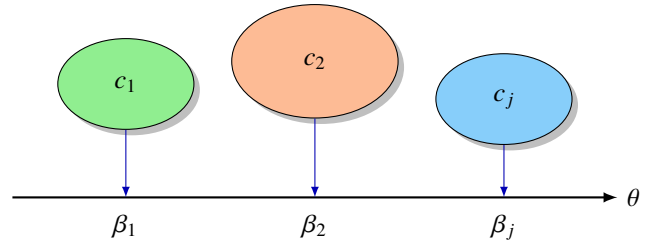


Figure 1: Visualization of the One-vs-All (OvA) classification strategy where multiple classes ($c_1, c_2, \dots, c_j$) are projected onto a single discriminant vector θ .

2. **One-vs-One (OvO) approach:** This strategy decomposes the multi-class problem into multiple binary classification problems. A common variant is One-vs-One (OvO) where c separate discriminant vectors $\{\theta_1, \dots, \theta_c\}$ are constructed. For each j^{th} class, a discriminant vector θ_j is found such that samples of this class project to one value (e.g., 1), while samples from all other classes project to a different value (e.g., 0). This is achieved by setting c different target vectors $\beta^1, \dots, \beta^c$. For θ_j , the corresponding target vector $\beta^j \in \mathbb{R}^n$ is defined as:

$$(\beta^j)_i = \begin{cases} 1 & \text{if sample } i \text{ belongs to class } j \\ 0 & \text{if sample } i \text{ does not belong to class } j \end{cases}$$

equivalently,

$$\beta^j = (\underbrace{0, \dots, 0}_{n_1}, \dots, \underbrace{1, \dots, 1}_{n_j}, \dots, \underbrace{0, \dots, 0}_{n_c})^T. \quad (5)$$

This is visualized conceptually in Figure 2, showing multiple discriminant vectors separating the classes.

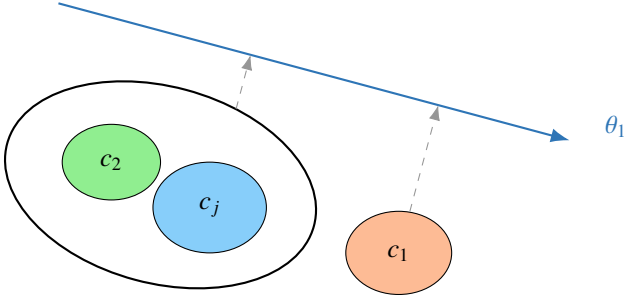


Figure 2: Two-dimensional representation of the One-vs-One (OvO) classification strategy showing how multiple classes ($c_1, c_2, \dots, c_j$) are separated using multiple discriminant vectors ($\theta_1, \dots, \theta_j$).

The use of the inner products in $Q^T Q$ suggests the possibility of kernelization, which will be detailed in Section 2.2.

2.1.2. Implementation algorithms

Based on the theory analysis provided in Section 2.1.1, Algorithm 1 and Algorithm 2, respectively, detail the procedures for obtaining the discriminant directions for the OvA strategy and OvO strategies.

The Algorithm 1 outlines a One-vs-All classification strategy for training a discriminant model. The input to the algorithm is the data matrix Q , and the output is the discriminant vector θ , which serves as the model's classifier. Initially, the algorithm constructs a target vector β , which consists of c subgroups corresponding to different class labels, where each subgroup represents the target value of samples per class. The algorithm then proceeds by solving the linear equation $Q^T Q \alpha = \beta$ to compute the intermediate variable α . Once α is determined, the discriminant vector θ is obtained by multiplying Q with α . The final step is to return θ , which is used for projecting testing data into the discriminant space and performing classification.

Algorithm 1: One-vs-All

Input: Q : The data matrix.

Output: discriminant vector, θ .

1: Setting:

$$\beta = (\underbrace{1, \dots, 1}_{n_1}, \underbrace{j, \dots, j}_{n_j}, \underbrace{c, \dots, c}_{n_c})^T.$$

/* Finding the discriminant vector */

2: Solve the equation $(Q^T Q) \alpha = \beta$ for α .

3: Obtain θ from equation $\theta = Q \alpha$.

4: **return** θ .

The Algorithm 2 presented the implementation of the One-vs-One classification strategy to train a set of discriminant vectors θ . The algorithm takes the data matrix Q as input and returns the output consisting of multiple discriminant vectors

$\theta_1, \theta_2, \dots, \theta_c$, where each vector is designed to distinguish specific classes. In the initialisation phase, the algorithm constructs a target vector β^j , which includes subgroups labelled as 0, 1, representing a specific class (with the rest set to zero). Each subgroup has a size denoted by n_1, n_j , and n_c , which indicates the number of samples in each class. The algorithm then solves the linear equation $Q^T Q \alpha = \beta^j$ to compute the intermediate vector α , which is used to find the discriminant vector θ_j by multiplying Q with α . The final step returns the discriminant vector θ_j , which is used to classify one class against others. This approach differs from the One-vs-All method by focusing on building a discriminant vector for each pair of classes, and then allowing more precise multi-class classification.

Algorithm 2: One-vs-One

Input: Q : The data matrix.

Output: discriminant vectors $\theta = \theta_1, \theta_2, \dots, \theta_c$.

1: Setting:

$$\beta^j = (\underbrace{0, \dots, 0}_{n_1}, \underbrace{1, \dots, 1}_{n_j}, \underbrace{0, \dots, 0}_{n_c})^T.$$

/* Finding the discriminant vectors θ_j */

2: **for** i from 1 to c **do**

3: Solve the equation $(Q^T Q) \alpha = \beta^j$ for α .

4: Obtain θ_j from equation $\theta_j = Q \alpha$.

5: **end for**

6: **return** θ .

2.2. Nonlinear case - Application of kernel trick

The linear CPAI formulation presented in Section 2.1 relies on the assumption of linear independence of the sample vectors x^i , which implies $n \leq d$. Such assumptions may not hold in many real-world problems, particularly when $n > d$ or the underlying class structures are non-linear, limiting the applicability of the linear theory. To overcome these limitations and to allow for more flexibility in the proposed model, the kernel trick can be employed, allowing the analysis to be performed implicitly in a potentially higher-dimensional feature space $\mathcal{H}$. We map features implicitly to a kernel feature space $\mathcal{H}$ with a kernel function, $\kappa(x^i, x^j) = (\Psi(x^i), \Psi(x^j))$ where the map Ψ is called a feature map.

The data x^i are mapped into the feature space $\mathcal{H}$ though the nonlinear implicit mapping function Ψ , $x^i \mapsto \Psi(x^i)$. Analogous to the linear case (2), we seek a discriminant direction θ that lies within the span of the mapped training samples:

$$\theta = \alpha_1 \Psi(x^1) + \alpha_2 \Psi(x^2) + \dots + \alpha_n \Psi(x^n). \quad (6)$$

The discriminant vector satisfies, for a given β (similar to (4)),

$$(\Psi(x^i), \theta) = \beta_i, \quad \forall i = \overline{1, n}. \quad (7)$$

It is equivalent to the matrix form as follows:

$$K \alpha = \beta, \quad (8)$$

where K means to be generated by the dot product of all the training samples i.e.,

$$K = [\kappa(x^i, x^j)]_{i,j \in \{1, \dots, n\}}. \quad (9)$$

Recall that a strictly positive definite kernel function κ if and only if its associated kernel matrix K is a symmetric positive definite matrix (i.e. it has positive eigenvalues [15]). If K is positive definite (i.e., invertible, full rank), then the system (8) has a unique solution α for any given β . Common examples of kernels often used in machine learning that are typically strictly positive definite, defined on Euclidean space $\mathbb{R}^r$ relying on specific parameter choices include RBF and Laplacian. This ensures that, from a theoretical standpoint, a unique solution for α always exists.

Furthermore, the positive definiteness of K (ensuring invertibility) is crucial for the existence and uniqueness of the solution α . While Mercer’s theorem provides conditions for positive definiteness, numerical stability might still require regularization (e.g., adding a small diagonal term $K + \epsilon I$) if K is ill-conditioned or only positive definite. Henceforth, if there is no confusion, we will use *positive definite kernel functions* to encompass both strictly positive definite and positive definite kernel functions.

2.3. Heuristic for Automatic Gamma Selection

The performance of kernel-based methods is critically dependent on the RBF kernel’s bandwidth hyperparameter, γ . To avoid computationally prohibitive grid search, we propose an efficient data-driven heuristic. Our method estimates an optimal γ by relating it to the data’s local structure via the average distance to the nearest neighbors. We compute the median of these average distances, σ , which serves as a robust estimator of the local data scale. The gamma is then computed as $\gamma = 1/(2\sigma^2)$. This heuristic calibrates the kernel’s bandwidth to the intrinsic data geometry, enhancing the model’s practicality.

3. Experiments

This section outlines our experiments designed to address the following research questions:

- **RQ1:** How effectively does the CPAI model classify attacks under two training approaches?
- **RQ2:** How significantly do the training approach and kernel selection impact the classification performance?
- **RQ3:** How does the performance of the CPAI model compare to baseline models?

3.1. Datasets

To assess the efficacy of our proposed algorithm, we conduct a comprehensive evaluation across five widely-used datasets: BoT-IoT, IoTID20, ToN-IoT, N-BaIoT, and CIC-IoT2023.

These datasets are specifically designed to emulate the complexities of IoT network environments and encompass various attack traffic patterns. A concise overview of the utilised datasets is provided below:

BoT-IoT [16]: The BoT-IoT dataset, published by the Cyber Range Lab of UNSW Canberra, serves as an important resource for supporting research in IoT network security. This dataset comprises various network traffic data captured from IoT devices infected with diverse botnet malware strains. These malicious programs are known to initiate a wide spectrum of cyberattacks, encompassing distributed denial of service (DDoS), brute-force incursions, and reconnaissance scans. The network traffic data acquisition took place within a meticulously controlled environment that replicates real-world IoT network configurations. Through comprehensive analysis, a comprehensive dataset of over 20 features was extracted from the raw network traffic, offering insights into key network parameters, flow dynamics, and traffic volume.

IoTID20 [17]: The IoTID20 dataset is well-designed to facilitate the development of IDSs for the IoT environment. It simulates real-world network traffic in an IoT setup and provides a foundation for identifying and mitigating malicious activities. IoTID20 covers a wide range of cyberattacks that specifically target IoT devices and networks. These include Distributed Denial of Service (DDoS), man-in-the-middle (MITM), data exfiltration, and scanning attacks.

ToN-IoT [18]: The ToN-IoT dataset, provided by the Cyber Range Lab of UNSW Canberra, is a resource designed to facilitate the development of artificial intelligence-driven security solutions tailored for industrial IoT networks. Unlike the BoT-IoT dataset, which focuses on attack traffic, the ToN-IoT dataset primarily encompasses benign network traffic generated by legitimate IoT devices operating under normal, non-compromised conditions. Furthermore, the ToN-IoT dataset also provides diversity, encompassing a wide range of IoT applications and communication protocols commonly deployed in real-world IIoT environments.

N-BaIoT [19]: The N-BaIoT dataset is a valuable resource for analysing the normal behavioural patterns of different IoT devices, encompassing sensors, actuators, and smart home appliances. By capturing the network traffic generated by these devices under benign conditions, the dataset establishes a baseline for comparison, aiding in identifying malicious activity. Moreover, the N-BaIoT dataset is notable for covering a wide range of IoT applications and communication protocols. This diversity ensures that the dataset is representative of real-world IoT deployments, enhancing its relevance and applicability to security research. The N-BaIoT dataset also includes instances of botnet attacks originating from two prominent botnet families: Mirai and Bashlite. These attacks provide valuable insights into the tactics and techniques employed by malicious actors seeking to exploit vulnerabilities in IoT devices.

CIC-IoT2023 [20]: The CIC-IoT2023 dataset, published by the Canadian Institute for Cybersecurity (CIC), represents a comprehensive resource designed to evaluate the performance and effectiveness of IDSs specifically tailored for IoT environments. This dataset includes a diverse range of IoT devices and

simulates many cyberattacks that pose significant threats to IoT ecosystems. For the aim of this research, the attacks contained within the CIC-IoT2023 dataset have been systematically categorized into five distinct attack categories: DDoS/DoS, scan, web attacks, spoofing, and botnet attacks.

3.2. Evaluation metrics

To evaluate the detection performance of our proposed algorithm in classification cyber attacks, we use a comprehensive set of benchmarking metrics calculated based on the confusion matrix. These include Accuracy (ACC), True Positive Rate (TPR), Positive Predictive Value (PPV), and False Positive Rate (FPR), which are commonly used to evaluate model performance. While these metrics provide an overall assessment of correctness, their reliability can be compromised in imbalanced datasets. To address this limitation, the F1-score, which balances precision and recall, and the Matthews Correlation Coefficient (MCC), which considers all four confusion matrix components, are incorporated to provide a more robust evaluation. We describe the formula for each metric below:

- Accuracy (ACC):

$$ACC = \frac{TP + TN}{TP + TN + FP + FN}$$

- Recall - True Positive Rate (TPR):

$$TPR = \frac{TP}{TP + FN}$$

- False Alarm Rate - False Positive Rate (FPR):

$$FPR = \frac{FP}{FP + TN}$$

- Precision - Positive Predictive Value (PPV):

$$PPV = \frac{TP}{TP + FP}$$

- F1-score:

$$F1\text{-score} = \frac{2 * TPR * PPV}{TPR + PPV}$$

- Matthews Correlation Coefficient (MCC):

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

3.3. Baseline competitive models

In this research, we employ a comprehensive set of baseline algorithms to evaluate our proposed approach. These baselines are machine learning algorithms based on discriminant analysis approaches including Gaussian Naive Bayes (GauNB), Kernel Null Foley-Sammon Transformation (KNFST), K-Nearest-Neighbour (KNN), Linear Discriminant Analysis (LDA), Nearest Centroid (NC), Support Vector Machine (SVM), Radius

Neighbors Classifier (RNC) or Stochastic Gradient Descent-SVM (SGD). These algorithms were specifically selected for their proven robustness in handling high-dimensional data and their widespread use in network intrusion detection. For implementation, we utilize the Sklearn Python library for most models, with the exception of KNFST, for which we provide our own implementation based on the original algorithm [14], due to the unavailability of official source code. Through this diverse set of baseline models, we aim to thoroughly evaluate our approach against established methods and highlight our effectiveness in addressing the research problem.

3.4. Multiclass classification performance under two training approaches with different kernel

This subsection presents the comprehensive experimental results to solve **RQ1** and **RQ2**. Our analysis is structured in two parts: First, Tables 1 and 2, respectively, summarize the comparative performance of various kernels with two proposed approaches. These results indicate the high accuracy of CPAI in classifying attacks in IoT networks. Second, Figures 3 and 4 provide a deeper comparative analysis, specifically highlighting the mean MCC and mean F1-score across the different kernels between the two proposed training approaches. The results reveal the significant impact on choosing training approach and selecting kernel. The detailed results are presented as follows.

3.4.1. The classification performance using different kernels

Table 1 highlights the remarkable accuracy achieved by the CPAI when trained with the One-vs-All (OvA) strategy. The experiment results reveal that three positive definite kernels, including Abel, Laplacian, and RBF, alternately achieve the top accuracy, but none of the kernels dominates the others. Particularly, the CPAI-OvA approach consistently achieves the highest accuracy from 95.41% across all benchmark datasets. Peak performance is observed on the N-BaIoT and BoT-IoT datasets, with accuracies of 99.42% and 99.49%, respectively, using the kernel Laplacian. The lowest accuracy, though still high, is recorded in the CIC-IoT2023 dataset, at 95.41%. While the Laplacian kernel achieves the highest accuracy in all datasets, it is surpassed by the kernel Abel and RBF in terms of TPR. The inequivalence between TPR and ACC suggests that Laplacian may be sensitive to class imbalance and biased by the majority class. Additionally, the Abel and Rbf kernels also alternately archive the best TPR score across benchmarked datasets. In contrast, non-positive definite kernels such as Sigmoid and Linear exhibit significantly lower ACC and TPR, along with higher False Positive Rates (FPR), compared to most positive definite kernels.

Table 2 highlights the superior accuracy achieved by the CPAI when trained with the One-vs-One (OvO) strategy. The experiment results indicate that the outstanding Laplacian kernel surpasses other kernels in all benchmarked datasets. Notably, the CPAI-OvO approach consistently surpasses 98.42% accuracy across all benchmark datasets when employing the Laplacian kernel. The peak performance is observed on the N-BaIoT and BoT-IoT datasets, achieving accuracies of 99.96%

Table 1: Performance evaluation of CPAI’s One-vs-All (OvA) approach across evaluation datasets, comparing different kernel settings using accuracy (ACC), true positive rate (TPR), and false positive rate (FPR) metrics. The best score is highlighted with red colour while the second best is highlighted with blue colour.

Kernel	BoT-IoT			IoTID20			ToN-IoT			N-BaIoT			CIC-IoT2023		
	ACC	TPR	FPR	ACC	TPR	FPR	ACC	TPR	FPR	ACC	TPR	FPR	ACC	TPR	FPR
None	98.39	94.48	0.97	82.86	62.00	10.71	88.18	51.78	6.76	96.93	86.17	1.73	91.12	57.94	5.33
Abel	99.24	97.94	0.45	95.24	88.47	2.98	98.02	80.50	1.11	98.97	85.53	0.57	95.21	69.70	2.79
Laplacian	99.49	84.26	0.30	97.57	77.78	1.46	98.23	81.44	1.00	99.42	87.54	0.32	95.41	69.41	2.68
Linear	98.32	94.35	1.01	83.64	60.42	10.23	89.48	56.31	6.01	95.58	80.11	2.49	91.63	61.33	5.02
Poly	99.23	97.25	0.46	88.18	71.98	7.39	95.92	84.21	2.33	98.94	95.22	0.60	93.28	69.99	4.03
Rbf	99.37	97.91	0.38	90.12	76.88	6.18	96.73	86.47	1.87	99.40	97.28	0.34	93.53	70.67	3.88
Sigmoid	92.36	78.70	4.58	78.20	43.51	13.63	87.20	45.75	7.31	90.57	57.56	5.31	85.68	39.44	8.59
Sobolev	95.03	88.73	2.98	78.92	52.09	13.18	89.67	62.88	5.90	92.70	63.96	4.10	91.67	50.29	5.21

Table 2: Performance evaluation of CPAI’s One-vs-One (OvO) approach across evaluation datasets, comparing different kernel settings using accuracy (ACC), true positive rate (TPR), and false positive rate (FPR) metrics. The best score is highlighted with red colour while the second best is highlighted with blue colour.

Kernel	BoT-IoT			IoTID20			ToN-IoT			N-BaIoT			CIC-IoT2023		
	ACC	TPR	FPR	ACC	TPR	FPR	ACC	TPR	FPR	ACC	TPR	FPR	ACC	TPR	FPR
None	99.51	98.52	0.30	91.08	78.31	5.58	98.23	91.75	1.01	98.38	93.57	0.91	97.25	86.57	1.65
Abel	99.62	98.59	0.23	95.98	89.91	2.51	99.05	95.80	0.54	99.95	99.78	0.03	97.93	89.89	1.24
Laplacian	99.77	99.14	0.14	98.42	96.05	0.99	99.24	96.96	0.43	99.96	99.84	0.02	98.83	94.25	0.70
Linear	99.54	98.48	0.28	92.94	82.99	4.41	98.24	92.06	1.00	96.90	81.03	1.74	97.90	89.71	1.26
Poly	99.37	97.60	0.38	93.34	84.53	4.16	98.45	93.26	0.89	99.78	99.00	0.13	98.32	91.86	1.01
Rbf	99.62	98.59	0.23	94.64	86.52	3.35	99.06	95.97	0.54	99.86	99.40	0.08	98.27	91.57	1.04
Sigmoid	97.02	90.88	1.79	86.76	66.23	8.28	95.13	77.97	2.78	94.81	77.85	2.92	91.02	53.23	5.39
Sobolev	99.41	97.95	0.36	95.74	90.29	2.66	98.70	94.59	0.74	99.69	98.64	0.17	96.93	85.99	1.84

and 99.77%, respectively. The lowest accuracy is recorded in the IoTID20 dataset, at 98.42%, which is close to the highest accuracy. Furthermore, the CPAI-OvO-Laplacian approach demonstrates stability, consistently achieving the best TPR while maintaining the lowest FPR across all datasets. Beyond the impressive accuracy of the Laplacian kernel, the Abel and RBF kernels alternately achieve the second-best accuracy and TPR on most benchmarked datasets. Additionally, the experiment results reveal that non-positive definite kernels, such as Sigmoid and Linear, exhibit significantly lower ACC and TPR, along with higher FPR, compared to most positive definite kernels.

3.4.2. The classification performance comparison between two training approach

Figures 3 and 4 show that the CPAI-OvO approach consistently outperforms the CPAI-OvA approach across all kernels in terms of MCC and F1-score. Specifically, while both approaches achieve the best MCC score using the Laplacian kernel, the CPAI-OvO significantly surpasses the OvA approach, with a score of 0.97 compared to 0.91, respectively. The largest gap was presented by the Sobolev kernel, with a difference of approximately 0.34, while the smallest gap was presented by the Abel kernel, with a difference of 0.05. This trend of OvO superiority is reflected in terms of F1-score, where CPAI-OvO surpasses CPAI-OvA by the largest gap of approximately 37% and the smallest gap of 9.8%. This result indicates the superiority of the OvO approach over the OvA approach in terms of classification performance.

Besides selecting training approaches, Figures 3 and 4 also indicate the significant impact of choosing the kernel. In general, selecting kernel has a significant effect on accuracy

and consistently affects both training strategies similarly. The positive-define kernels, such as Abel, Laplacian, Rbf give higher accuracy than non-positive-define kernels like Linear and Sigmoid kernels. While the Laplacian archives the best MCC with 0.97 when trained with the OvO approach, the Sigmoid kernel only reaches an MCC of 0.72. This trend is similar to the OvA approach. Besides, applying positive-define kernels also gives an improvement in accuracy compared to not using kernels. However, the results also reveal that the non-positive-define kernels tend to give worse classification performance than not use kernels. These findings collectively highlight the crucial role of kernel selection in enhancing model performance.

3.5. Baseline comparison results

This subsection presents a comprehensive comparative analysis of our proposed algorithm against widely used baseline models on different evaluation metrics, directly answering the **RQ3**. The tables 3 to 7 presents the comparison results on dataset Bot-IoT, IoTID20, ToN-IoT, N-BaIoT and CIC-IoT2023, respectively. To facilitate interpretation, we highlight the highest-performing algorithm in each metric with red text and the second-best performer with blue text. The analysis demonstrates that our proposed algorithm consistently achieves superior performance across multiple metrics compared to the baseline approaches.

For more details, training the proposed algorithm under the OvO strategy consistently yields the highest and most stable accuracy across all benchmark datasets. The algorithm reaches its peak on the N-BaIoT dataset with MCC of 0.998, while maintaining a minimum MCC value exceeding 0.95. It also demonstrates stability by keeping the ACC above 98% in all datasets.

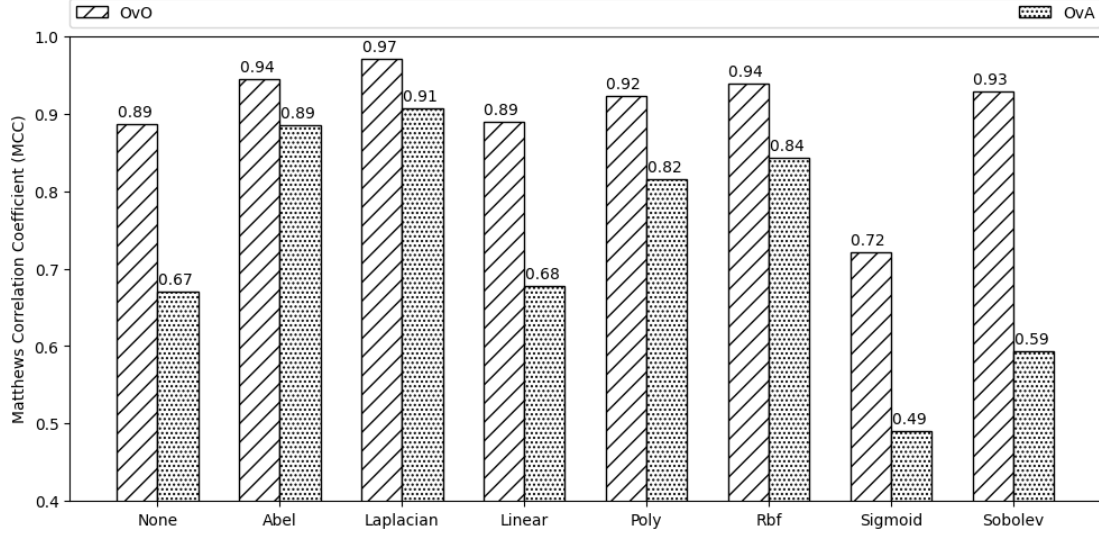


Figure 3: Comparative analysis of mean MCC score using different kernel functions between two CPAI’s classification strategy approaches.

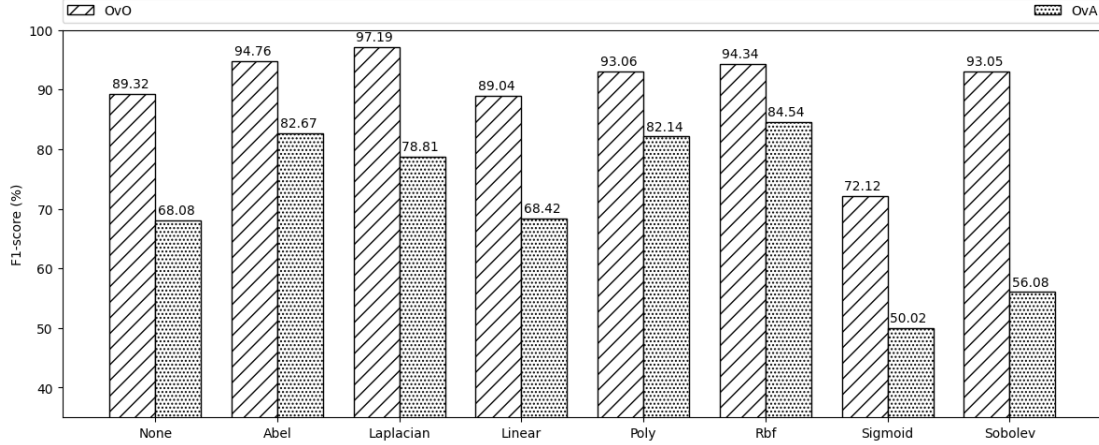


Figure 4: Comparative analysis of mean F1-score using different kernel functions between two CPAI’s classification strategy approaches.

Similarly, the CPAI-OvO approach achieves high scores across other metrics, surpassing 94% on all datasets, while keep the false positive rate remains consistently below 1%, lower than other competitors.

Beyond superior accuracy, CPAI-OvO flexibly adapts to different environments by reaching the first rank in all evaluated datasets. Specifically, the algorithm can accurately classify different attacks across all benchmark datasets, maintaining an MCC above 0.95. In contrast, the KNN algorithm achieved second place on only two datasets, while KNFST, and SVM sequence attained second positions on BoT-IoT, and CIC-IoT2023, respectively, but faltered on other datasets.

In conclusion, the comparison results highlight the outstanding of the proposed algorithm in classifying cyber attacks in the IoT networks. The CPAI-OvO consistently provides high accuracy while maintaining stability across different environments.

4. Conclusion and perspectives

In conclusion, this paper introduced CPAI, a novel discriminant analysis framework that resolves the critical singularity and SSS limitations of NFAST. By explicitly controlling projection targets and leveraging the kernel trick, CPAI provides a robust and flexible framework for multi-class classification. Our extensive experimental evaluation across five benchmark IoT datasets demonstrates CPAI’s superior performance, consistently achieving higher accuracy and stability compared to existing distance-based algorithms.

However, our approach has several limitations. First, the performance of Kernel CPAI is dependent on the selection of a suitable kernel and its hyperparameters. While our proposed gamma heuristic mitigates this, the regularization parameter λ may still require tuning for optimal performance. Second, the kernelized version’s training complexity scales with the number of samples (n), which can be a bottleneck for extremely large datasets. Future work could explore scalable kernel approxima-

Table 3: Comparative evaluation of CPAI (OvA and OvO variants) against eight baseline models (GauNB, KNFST, KNN, LDA, NC, SVM, RNC, SGD) on the BoT-IoT dataset using six performance metrics: Matthews Correlation Coefficient (MCC), Accuracy (ACC), True Positive Rate (TPR), False Positive Rate (FPR), Positive Predictive Value (PPV), and F1-score.

Model	MCC ↑	ACC ↑	TPR ↑	FPR ↓	PPV ↑	F1 ↑
GauNB	0.796	93.99	81.59	3.61	83.16	78.71
KNFST	0.983	99.52	98.41	0.29	98.07	98.23
KNN	0.978	99.41	97.71	0.36	98.06	97.88
LDA	0.950	98.62	95.29	0.83	95.27	95.26
NC	0.703	91.63	64.73	5.02	72.74	61.35
SVM	0.916	97.68	93.02	1.39	92.61	92.68
RNC	0.925	97.89	92.83	1.26	95.59	93.78
SGD	0.976	99.34	97.75	0.40	97.65	97.69
CPAI-OvA	0.978	99.49	84.26	0.30	83.85	84.04
CPAI-OvO	0.992	99.77	99.14	0.14	99.19	99.17

Table 4: Comparative evaluation of CPAI (OvA and OvO variants) against eight baseline models (GauNB, KNFST, KNN, LDA, NC, SVM, RNC, SGD) on the IoTID20 dataset using six performance metrics: Matthews Correlation Coefficient (MCC), Accuracy (ACC), True Positive Rate (TPR), False Positive Rate (FPR), Positive Predictive Value (PPV), and F1-score.

Model	MCC ↑	ACC ↑	TPR ↑	FPR ↓	PPV ↑	F1 ↑
GauNB	0.505	81.06	52.65	11.84	77.73	49.06
KNFST	0.654	88.90	72.25	6.94	71.98	71.96
KNN	0.754	92.12	80.30	4.93	80.25	80.23
LDA	0.795	93.38	83.45	4.14	84.19	83.48
NC	0.432	81.50	53.75	11.56	52.45	49.41
SVM	0.788	93.20	83.00	4.25	83.62	83.13
RNC	0.419	78.46	46.15	13.46	67.09	43.57
SGD	0.653	87.28	68.20	7.95	80.57	67.08
CPAI-OvA	0.852	95.24	88.47	2.98	88.10	88.12
CPAI-OvO	0.951	98.42	96.05	0.99	96.05	96.04

Table 5: Comparative evaluation of CPAI (OvA and OvO variants) against eight baseline models (GauNB, KNFST, KNN, LDA, NC, SVM, RNC, SGD) on the ToN-IoT dataset using six performance metrics: Matthews Correlation Coefficient (MCC), Accuracy (ACC), True Positive Rate (TPR), False Positive Rate (FPR), Positive Predictive Value (PPV), and F1-score.

Model	MCC ↑	ACC ↑	TPR ↑	FPR ↓	PPV ↑	F1 ↑
GauNB	0.277	83.53	38.21	9.41	33.55	27.32
KNFST	0.896	97.75	90.76	1.29	89.72	89.87
KNN	0.943	98.79	94.35	0.69	94.89	94.59
LDA	0.908	98.03	92.06	1.12	90.94	91.43
NC	0.259	82.66	39.43	9.91	40.41	25.33
SVM	0.875	97.32	87.33	1.53	89.27	88.00
RNC	0.871	97.15	86.58	1.63	93.78	88.91
SGD	0.829	96.30	83.57	2.11	85.50	83.64
CPAI-OvA	0.907	98.23	81.44	1.00	81.38	81.32
CPAI-OvO	0.965	99.24	96.96	0.43	96.61	96.76

tion techniques, such as the Nyström method, to address this. Finally, the OvA strategy, while fast, can be sensitive to severe class imbalance.

5. Acknowledgment

This research was supported by the VNUHCM-University of Information Technology’s Scientific Research Support Fund.

References

- [1] A. Aldhaheri, F. Alwahedi, M. A. Ferrag, and A. Battah, “Deep learning for cyber threat detection in iot networks: A review,” *Internet of Things and cyber-physical systems*, vol. 4, pp. 110–128, 2024.
- [2] A. Heidari and M. A. Jabraeil Jamali, “Internet of things intrusion detection systems: a comprehensive review and future directions,” *Cluster Computing*, vol. 26, no. 6, pp. 3753–3780, 2023.
- [3] A. Jamalipour and S. Murali, “A taxonomy of machine-learning-based intrusion detection systems for the internet of things: A survey,” *IEEE Internet of Things Journal*, vol. 9, no. 12, pp. 9444–9466, 2021.
- [4] B. K. Mohanta, D. Jena, U. Satapathy, and S. Patnaik, “Survey on iot security: Challenges and solution using machine learning, artificial intelli-

Table 6: Comparative evaluation of CPAI (OvA and OvO variants) against eight baseline models (GauNB, KNFST, KNN, LDA, NC, SVM, RNC, SGD) on the **N-BaIoT** dataset using six performance metrics: Matthews Correlation Coefficient (MCC), Accuracy (ACC), True Positive Rate (TPR), False Positive Rate (FPR), Positive Predictive Value (PPV), and F1-score.

Model	MCC ↑	ACC ↑	TPR ↑	FPR ↓	PPV ↑	F1 ↑
GauNB	0.721	94.25	74.11	3.24	75.22	71.26
KNFST	0.960	99.20	96.39	0.45	96.69	96.36
KNN	0.984	99.68	98.56	0.18	98.57	98.55
LDA	0.883	97.62	89.28	1.34	90.39	87.82
NC	0.450	88.54	48.44	6.44	38.95	39.74
SVM	0.976	99.53	97.89	0.26	97.92	97.89
RNC	0.697	93.37	70.17	3.73	89.14	72.62
SGD	0.912	98.22	92.00	1.00	92.67	91.79
CPAI-OvA	0.969	99.40	97.28	0.34	97.32	97.27
CPAI-OvO	0.998	99.96	99.84	0.02	99.83	99.83

Table 7: Comparative evaluation of CPAI (OvA and OvO variants) against eight baseline models (GauNB, KNFST, KNN, LDA, NC, SVM, RNC, SGD) on the **CIC-IoT2023** dataset using six performance metrics: Matthews Correlation Coefficient (MCC), Accuracy (ACC), True Positive Rate (TPR), False Positive Rate (FPR), Positive Predictive Value (PPV), and F1-score.

Model	MCC ↑	ACC ↑	TPR ↑	FPR ↓	PPV ↑	F1 ↑
GauNB	0.790	94.95	75.48	3.03	75.45	72.71
KNFST	0.836	96.17	81.17	2.30	80.97	81.00
KNN	0.858	96.67	83.47	2.00	83.88	83.48
LDA	0.773	94.68	73.55	3.19	74.29	73.46
NC	0.492	85.78	55.72	8.53	59.23	48.96
SVM	0.884	97.30	86.57	1.62	88.78	87.49
RNC	0.538	89.63	48.17	6.22	50.06	46.75
SGD	0.781	94.80	74.07	3.12	76.92	73.57
CPAI-OvA	0.771	95.41	69.41	2.68	63.07	63.82
CPAI-OvO	0.950	98.83	94.25	0.70	94.17	94.14

- gence and blockchain technology,” *Internet of Things*, vol. 11, p. 100227, 2020.
- [5] R. Ahmad, I. Alsmadi, W. Alhamdani, and L. Tawalbeh, “A comprehensive deep learning benchmark for iot ids,” *Computers & Security*, vol. 114, p. 102588, 2022.
- [6] Z. Azam, M. M. Islam, and M. N. Huda, “Comparative analysis of intrusion detection systems and machine learning based model analysis through decision tree,” *IEEE Access*, 2023.
- [7] X.-H. Nguyen, X.-D. Nguyen, H.-H. Huynh, and K.-H. Le, “Realguard: A lightweight network intrusion detection system for iot gateways,” *Sensors*, vol. 22, no. 2, p. 432, 2022.
- [8] F. Falcão, T. Zoppi, C. B. V. Silva, A. Santos, B. Fonseca, A. Ceccarelli, and A. Bondavalli, “Quantitative comparison of unsupervised anomaly detection algorithms for intrusion detection,” in *Proceedings of the 34th ACM/SIGAPP symposium on applied computing*, pp. 318–327, 2019.
- [9] R. A. FISHER, “The use of multiple measurements in taxonomic problems,” *Annals of Eugenics*, vol. 7, no. 2, pp. 179–188, 1936.
- [10] Y.-F. Guo, L. Wu, H. Lu, Z. Feng, and X. Xue, “Null foley–sammon transform,” *Pattern Recognition*, vol. 39, no. 11, pp. 2248–2251, 2006.
- [11] D. H. Foley and J. W. Sammon, “An optimal set of discriminant vectors,” *IEEE Transactions on Computers*, vol. C-24, pp. 281–289, 1975.
- [12] T. Ali and S. Chaudhuri, “Theoretical analysis of null foley–sammon transform and its implications,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, pp. 6445–6459, may 2023.
- [13] Y. Lin, G. Gu, H. Liu, and J. Shen, “Kernel null foley–sammon transform,” in *2008 International Conference on Computer Science and Software Engineering*, vol. 1, pp. 981–984, 2008.
- [14] P. Bodesheim, A. Freytag, E. Rodner, M. Kemmler, and J. Denzler, “Kernel null space methods for novelty detection,” in *2013 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3374–3381, 2013.
- [15] H. Anton and C. Rorres, *Elementary Linear Algebra: Applications Version, 11th Edition*. John Wiley & Sons Incorporated, 2013.
- [16] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, “Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-iot dataset,” *Future Generation Computer Systems*, vol. 100, pp. 779–796, 2019.
- [17] I. Ullah and Q. H. Mahmoud, “A scheme for generating a dataset for anomalous activity detection in iot networks,” in *Canadian conference on artificial intelligence*, pp. 508–520, Springer, 2020.
- [18] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, “Ton.iot telemetry dataset: A new generation dataset of iot and iiot for data-driven intrusion detection systems,” *Ieee Access*, vol. 8, pp. 165130–165150, 2020.
- [19] Y. Meidan, M. Bohadana, Y. Mathov, Y. Mirsky, A. Shabtai, D. Breitenbacher, and Y. Elovici, “N-baiot—network-based detection of iot botnet attacks using deep autoencoders,” *IEEE Pervasive Computing*, vol. 17, no. 3, pp. 12–22, 2018.
- [20] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, “Ciciot2023: A real-time dataset and benchmark for large-scale attacks in iot environment,” *Sensors*, vol. 23, no. 13, p. 5941, 2023.

Appendix A. Overview of NFST and KNFST

In this appendix, we provide a brief overview of NFST and its kernelized version (KNFST) to make the paper self-contained and to provide context for the proposed CPAI method. Assume dataset consists of n samples (denoted by x^i) belonging to c classes $\{x_1^i\}_{i=1}^{n_1}, \dots, \{x_c^i\}_{i=1}^{n_c}$ where n_j and $x_j^i \in \mathbb{R}^d$ denote the sample size and the i^{th} instance of the j^{th} class, respectively. Let us define $\bar{x}_j = \frac{1}{n_j} \sum_{i=1}^{n_j} x_j^i$ and $\bar{x} = \frac{1}{n} \sum_{i=1}^n x^i$ the mean of

the j^{th} class and the total mean of the data, respectively. Foley-Sammon Transform (FST) [11] aims to find an optimal discriminant vector $\varphi \in \mathbb{R}^d$ that maximizes the Fisher criterion [9], under orthonormal constraints:

$$J(\varphi) = \frac{\varphi^T S_b \varphi}{\varphi^T S_w \varphi}, \quad (\text{A.1})$$

where $S_w \in \mathbb{R}^{d \times d}$ and $S_b \in \mathbb{R}^{d \times d}$ are the within-class scatter matrix defined and the between-class scatter matrix, respectively defined by

$$S_w := \sum_{j=1}^c \sum_{i=1}^{n_j} (x_j^i - \bar{x}_j)(x_j^i - \bar{x}_j)^T$$

$$S_b := \sum_{j=1}^c n_j (\bar{x}_j - \bar{x})(\bar{x}_j - \bar{x})^T.$$

If S_w is nonsingular, φ_1 is the unit eigenvector corresponding to the maximum eigenvalue of $S_w^{-1} S_b$. The remaining discriminant directions are obtained by solving

$$\max_{\varphi_i} J(\varphi) \quad \text{s.t.}, \quad \varphi_i^T \varphi = \delta_{ij}, \quad j = 1, \dots, i-1. \quad (\text{A.2})$$

The FST is then $y = \Phi^T x$ where $\Phi = (\varphi_1, \varphi_2, \dots, \varphi_r)$.

In contrast to FST, Null Foley-Sammon Transform (NFST) [10] aims to find φ -so-called null projecting direction (NPD), which satisfies

$$\varphi^T S_w \varphi = 0, \quad (\text{A.3})$$

$$\varphi^T S_b \varphi > 0. \quad (\text{A.4})$$

Then the data have the best separability on the direction φ according to the Fisher discriminant criterion. For the small sample size case i.e., $n \leq d$, we additionally assume that the sample vectors are independent. Let us define $Z_t = \{x \in \mathbb{R}^d \mid S_t x = 0\}$ where $S_t = S_w + S_b$ is the total scatter matrix, and Z_t^T denotes Z_t 's the orthogonal complement space. It can be shown that

Theorem 3. (see [10]) *If the sample vectors are independent, there are $c - 1$ NPDs $\varphi_i \in (Z_w \cap Z_t^\perp)$ where $Z_w = \{x \in \mathbb{R}^d \mid S_w x = 0\}$.*

The standard algorithm for computing NFST is summarized as follows

- Step 1. Compute the orthonormal basis $\tau_1, \dots, \tau_{n-1}$ of the subspace $Z_t^\perp$ using the Gram-Schmidt orthogonalization procedure.
- Step 2. Denote $S = P^T S_w P \in \mathbb{R}^{(n-1) \times (n-1)}$ where $P = (\tau_1, \dots, \tau_{n-1})$. Solving the problem $S\eta = 0$, we can obtain a set of vectors $\eta_1, \dots, \eta_{c-1}$.
- Step 3. The discriminant vectors are $\varphi_i = P\eta_i, i = 1, c-1$.
- Step 4. NFST is then given by

$$y = \Phi^T x, \quad \text{where } \Phi = (\varphi_1, \dots, \varphi_{c-1}).$$

To make it efficient for $d > n$ case. The nonlinear version of the theory was proposed; see the details in [13]. The key point

is to take each column vector of K as a sample vector. Here, K means to be generated by the dot product of all the training samples in the implicit feature space, which is similar to (9). The procedure of computing KNFST can be summarized in the following steps [13]:

- Step 1. Compute the kernel matrix K using (9).
- Step 2. Compute the orthogonal basis $(\tau_1, \dots, \tau_{n-c})$ from the set of vectors $\{K_{\cdot 1} - \bar{k}, K_{\cdot 2} - \bar{k}, \dots, K_{\cdot n} - \bar{k}\}$ where $\bar{k} = \frac{1}{n} \sum_{i=1}^n K_{\cdot i}$.
- Step 3. Set $P = (\tau_1, \dots, \tau_{n-c})$, compute $S = P^T K(I - L)$ where I is the $n \times n$ identity matrix, and $L = (L_l)_{l=1, \dots, c}$ is a $n \times n$ block diagonal matrix, L_l is a $n_l \times n_l$ matrix with all terms equal to $1/n_l$.
Solving the problem $S\eta = 0$, we can obtain a set of vectors $\eta_1, \dots, \eta_{c-1}$.
- Step 4. KNFST is then given by AQ , where $A = (\eta_1, \dots, \eta_{c-1})$.

The samples of each class get merged into a single point. However, Ali et al.[12] showed that the singularities of different classes are not necessarily always distinct, losing their class identities, and there would be no solution for NFST based classifiers.

Appendix B. In comparison with NFST

This study was motivated by the identified shortcomings of NFST. It is reasonable to make a comparison between the present approach and NFST and its related studies to highlight the key differences and advantages of the proposed CPAI approach. To make it easy for the readers, we refer the readers to Appendix A and references therein for details about NFST.

1. **Addressing the Singularity Problem:** NFST relies on the constraints $\varphi^T S_w \varphi = 0$ and $\varphi^T S_b \varphi > 0$. While aiming for distinct singularities, it offers no explicit control over their final locations in the projection space. As shown by [12], these singularities can merge under certain conditions. In contrast, CPAI directly addresses this by allowing the user to *predefine* distinct target projection values (β_j) for each class. The algorithm then explicitly constructs discriminant vectors (θ or functions via α) to map samples towards these predefined, separated targets (solving $Q^T Q \alpha = \beta$ or $K \alpha = \beta$). This explicit control over target points inherently prevents the merging of class singularities, which is a primary contribution of CPAI.
2. **Handling Data Dimensionality and Assumptions:**
 - Linear NFST fundamentally requires linear independence of sample vectors, which generally necessitates $n \leq d$. This limits its direct application in common scenarios where $n > d$.
 - Linear CPAI, as formulated through Theorem 1, also requires linear independence for the uniqueness proof via the invertibility of $Q^T Q$. It is thus similarly constrained in the $n > d$ linear case.

- However, Kernel CPAI directly overcomes the $n \leq d$ limitation by working with the $n \times n$ kernel matrix K . It solves $(K + \epsilon I)\alpha = \beta$. As long as K is invertible (which is true for strictly positive definite kernels and is often true for appropriate positive definite kernels, possibly with regularization), a unique solution α exists regardless of the relationship between n and d . It also implicitly handles non-linearities and does not require linear independence in the original space $\mathbb{R}^d$. This makes Kernel CPAI significantly more adaptable to diverse real-world datasets compared to linear NFST or linear CPAI. KNFST also addresses the $n > d$ issue via kernelization, but remains susceptible to the singularity problem that Kernel CPAI aims to solve through controlled targets.

3. **Implementation and Kernel Choice:** The effectiveness of Kernel CPAI relies on selecting a suitable kernel function κ that results in a well-conditioned (or regularized) and invertible kernel matrix K , and captures the underlying data structure well. Our experiments demonstrate that appropriate kernel choices (like Laplacian, RBF, Abel in our tests) significantly enhance performance over the linear case and potentially problematic kernels like Sigmoid. This highlights the importance of kernel selection, a common aspect of kernel methods.

In essence, while inspired by the discriminant analysis framework, CPAI differentiates itself from NFST/KNFST by offering explicit control over target projections to resolve the singularity problem and by leveraging the kernel trick more robustly to handle diverse data dimensionalities and non-linearities.